Should You Build a Deterministic Multi-CLI Orchestrator? An Honest Assessment and a Better-Scoped Design (October 2026)

Don't build the general orchestrator you described. By late 2026 the "run Claude Code, Codex, OpenCode and friends from one place, with auto-detection" space is crowded. What is still missing is a trustworthy, deterministic preflight-and-capability layer: something that can say which agents, models and auth modes will actually complete a headless turn right now, and resolve your role preferences against that. Build that as a small, sharp tool (a "doctor + resolver") that emits a machine-readable capability manifest. Add a thin declarative workflow runner only if you still need one after trying the existing tools.

TL;DR

- **Most of your idea already exists.** Vibe Kanban, Agent of Empires, Parallel Code, Crewplane, multicli, claude-code-router, Ruflo, and Claude Code's own subagents, agent teams and dynamic workflows already cover multi-CLI launching, auto-detection of installed CLIs, model routing with failover, and declarative multi-stage workflows. A generic orchestrator would be roughly the 30th entrant, with no moat.
- **The real gap is honest preflight.** The CLIs are inconsistent and sometimes misleading about health:
  - Claude Code's headless mode has a documented bug where a rate-limit error returns `subtype:"success"` with exit code 0. github
  - Codex has no headless way to read quota. github
  - Antigravity CLI (which replaced Gemini CLI for individual accounts on June 18, 2026) continuumcode has no auth-status command at all.

  A deterministic tool that normalizes this into "available / authenticated / functional / quota-limited", per CLI and per model, is useful and not well served today.
- **Recommendation: build `agent-doctor` first.** Make it a Go (or Rust) single binary with:
  - a per-CLI adapter interface;
  - tiered probes (presence → auth → live smoke test), cached with a TTL;

- one canonical YAML config validated by JSON Schema (JSON accepted for free);
- a deterministic rule-based role resolver with fallback chains.

Expose the manifest so existing runners (Crewplane, Vibe Kanban, your own scripts) can consume it. Only then consider a minimal workflow runner, and keep the planner non-AI.

Key Findings

1. The landscape: what already exists and what each does or doesn't do

Category	Tool	What it does	Auto-detect / health / fallback
Native multi-agent	Claude Code subagents, agent teams, dynamic workflows	Subagents report back to the parent. Agent teams coordinate multiple Claude Code instances through a shared task list and messaging; they are experimental, disabled by default, and don't isolate teammates in worktrees. Dynamic workflows move the plan into a script that the runtime executes, with up to 16 concurrent agents by default. claude claude	Claude-only. No cross-vendor health checks.
Session managers	Agent of Empires (Rust, tmux)	TUI and web dashboard. Each agent gets a tmux session, optional git worktree and optional Docker sandbox. Supports 16 agents, including Claude Code, OpenCode, Codex, Gemini, Antigravity, Cursor, Copilot, Pi and Kiro. agent-of-empires	"AoE auto-detects which are installed." Reports status (running/waiting/idle/error), not functional health. agent-of-empires
Session managers	Vibe Kanban (Rust)	Kanban board. Each task runs in its own worktree with a terminal and dev server. Supports 10+ agents, including Claude Code, Codex, Gemini CLI, Copilot, Amp, Cursor, OpenCode, Droid, CCR and Qwen Code. ai-tldr	Uses an executor abstraction per agent. Runs agents with permission-skipping flags by default. virtuslab elite-ai-assisted-cod…

Category	Tool	What it does	Auto-detect / health / fallback
Session managers	Parallel Code, Crystal, CliDeck, Claude Squad, CCManager, etc.	Parallel worktrees plus diff review. github github	Mostly presence-based.
Declarative workflow runners	Crewplane (Python, Apache-2.0)	Markdown workflows: YAML frontmatter defines a DAG and Markdown sections hold the prompts. Supports parallel fan-out across providers and executor/reviewer loops. Handoffs are persisted on disk, and a preflight step validates the composed DAG. Works with Claude Code, Codex, Gemini, Copilot CLI or any CLI. github	Validates the DAG, not provider health. Small project (about 41 stars). openorchestrators
Declarative workflow runners	oh-my-claudecode, Claude Octopus, Maestro-Orchestrate	Claude-hosted plugins that spawn Codex, Gemini or Antigravity workers or consensus reviewers. danielvaughan +2	Expect the CLIs to be “installed/authenticated.” github
Cross-CLI bridges	multicli (MCP)	Lets Claude, Antigravity, Codex and OpenCode call each other as tools. github	“Detects which AI CLIs you have installed and configures Multi-CLI for them automatically.” github

Category	Tool	What it does	Auto-detect / health / fallback
Swarm frameworks	Ruflo (formerly claude-flow)	Hive-mind orchestration, memory, and multi-provider routing (Claude, GPT, Gemini, Cohere, Ollama) with failover.	Heavy. Gerald Chen's May 11, 2026 plugin comparison says Ruflo "underdelivers in execution" and concludes it "is not recommended".
Model gateways	claude-code-router (musistudio)	A local gateway giving Claude Code, Codex, OpenCode, Kimi, Grok CLI and others "one stable local endpoint," with routing, failover and observability. github	Provider-level failover.
Model gateways	LiteLLM proxy	Fallbacks, retries, cooldowns (`allowed_fails`, `cooldown_time`), and background health-check-driven routing (`health_check_interval` default 300s).	Mature health and fallback, but at the API level, not the CLI level. litellm litellm
Model gateways	OpenRouter	A `models` array tried in priority order on downtime, rate limits or moderation refusals.	API-level only. openrouter
Standards	ACP (Agent Client Protocol)	JSON-RPC over stdio between a client and an agent. 42 agents in the official ACP registry as of October 2026, per Maksim Danilchenko's ACP guide (Claude via Zed's adapter, Codex via adapter, Gemini CLI, OpenCode, Goose, Cline, Copilot preview, etc.). Registry launched January	`initialize` returns `agentCapabilities` and `authMethods`. This is the closest thing to standardized capability discovery.

Category	Tool	What it does	Auto-detect / health / fallback
		28, 2026 by JetBrains and Zed, per both companies' announcement posts.	
Standards	MCP, AGENTS.md, A2A	MCP covers tools and data. AGENTS.md is, per its official site, "a README for agents" "used by over 60k open-source projects." A2A is agent-to-agent.	None of them address install or auth health.

What this means: launching multiple CLIs, isolating them in worktrees, detecting which ones are installed, and routing models with failover are all solved problems. Multiple tools do it, several with large followings: Vibe Kanban, for example, has about 28.4k GitHub stars, though its README now says it is sunsetting after Bloop shut down on April 10, 2026, and it continues as a community-maintained open-source project. Your feature list as described ("receive a pattern, validate, build a strategy from what works, use preferences or auto-detect") overlaps heavily with Crewplane plus Agent of Empires plus claude-code-router combined.

2. Where the real gaps are

The gaps all sit below the orchestrators. They assume that "installed" means "works," and it often doesn't.

a) Exit codes and status fields are unreliable. This is the big one.

- Claude Code headless:
  - Issue #79500: API errors such as rate limits return `"subtype":"success"` with `"is_error":true` and exit code 0. In plain text mode the error goes to stdout. github
  - Issue #74761: `-p` can exit 0 mid-task. github
  - Issue #103253 (v2.1.302, October 2026) is the reverse: a late `rate_limit_event` makes a successful run exit 1 with empty stdout in json/text mode, while `stream-json` is unaffected. github

- Codex: `codex doctor --json` emits a redacted machine-readable report, openai but bug reports show it exiting 0, so you must gate on the JSON contents rather than the exit code. github +2
- Gemini CLI uses specific exit codes (41 = `FatalAuthenticationError`). github It also has a known bug where setting `GOOGLE_GEMINI_BASE_URL` triggers exit 41 even with a valid API key. github

b) Auth-state detection differs per CLI, and some CLIs have none.

CLI	Non-interactive auth check	Machine-readable?	Exit code reflects login?	Gotchas
Claude Code	`claude auth status` (JSON by default; `--text` for humans) claude-wiki	Yes: `loggedIn`, `authMethod`, `configDirectory` github claude nousresearch blakecrosley	0 if logged in, 1 if not. But 1 is also returned for ordinary CLI errors. github claude-wiki	A reported bug: it could start an OAuth refresh and exit without saving it, causing later "token revoked" errors. The changelog says this is fixed. github claude Don't poll it in a loop.
Codex	`codex login status` and `codex doctor --json`	Login status: text only. Doctor: JSON.	Login status: 0 when credentials are present. codex-docs Doctor: not reliably.	A one-off `CODEX_API_KEY` used with `codex exec` doesn't count as a cached login. There is no headless quota view (issue #10233). github simplified
OpenCode	`opencode auth list --format json` (v2; `providers` in 1.x) opencode computingforgeeks	Yes	Not documented, so parse the output.	The command was renamed between v1 and v2. computingforgeeks
Antigravity (`agy`)	None documented	n/a	A headless run without cached credentials "exits	Only a live probe works. Check `.status ==`

CLI	Non-interactive auth check	Machine-readable?	Exit code reflects login?	Gotchas
			with an `authentication required` error instead of hanging" antigravity	`"SUCCESS"`; WAITING is not a failure. continuumcode Older builds lack `--output-format`. dev
Gemini CLI (legacy, paid/enterprise)	None. Probe and check for exit 41.	JSON `error` object github	Yes (41) github	Replaced for free and individual tiers. aibuilderclub

c) Model enumeration is inconsistent. Only OpenCode has a first-class `opencode models` command ("every model your credentials can reach"). computingforgeeks I found no model-listing command for Claude Code, Codex or Antigravity. That means your "preferred Claude primary/sub-agent/auxiliary model" must be validated by a live probe with `--model`, not by looking it up in a list.

d) Headless invocation differs per CLI, which is why adapters are mandatory.

CLI	Headless	Machine output	Permission / sandbox control	Resume
Claude Code	`claude -p`	`--output-format json` / `stream-json` (has `is_error`, `total_cost_usd`, `permission_denials`) anthropic dev	`--permission-mode`, `--allowedTools`; `--bare` skips repo hooks and `.mcp.json` and ignores the subscription login dev	`--continue`, `--resume <id>`
Codex	`codex exec`	`--json` (JSONL), `--output-schema`, `-o` openai onlinetool	Read-only sandbox by default; `--sandbox workspace-write` moodybeard dev	`codex exec resume --last` onlinetool
OpenCode	`opencode run`	`--format json` (NDJSON: step_start, text, tool_use, step_finish with tokens and cost) skills	`permission` in `opencode.json`, `--auto` dev	`-c`, `-s <id>`
Antigravity / Gemini	`agy -p` / `gemini -p`	`--output-format json` / `stream-json`; agy adds `--json-schema` antigravity dev continuumcode	Gemini: `--approval-mode` plus a policy engine; in headless mode, a rule that would ask "is treated as deny" dev	`--resume latest`
Copilot CLI	`copilot -p`	`--output-format json` (JSONL), `-s` dev	`--allow-tool`, `--deny-tool` dev	`--continue`, `--resume`

e) Quota awareness is mostly absent. Claude subscriptions use rolling 5-hour and weekly windows. dev +2 Claude Code reads rate-limit headers internally and, as of

v2.1.283, uses a small grace allowance to wrap up the current step, samuellawrentz but none of this is exposed as a stable preflight API. Codex exposes no quota headlessly. github The community response is a set of statusline and menu-bar tools that scrape usage. dev +2 So the honest best you can offer is reactive detection plus a cooldown, which is exactly the LiteLLM model (`allowed_fails` + `cooldown_time`) applied to CLIs. deepwiki swfte

f) Verification of results. Headless runs can "succeed" on an empty diff when tool calls are silently denied. dev A runner should verify outcomes deterministically (diff non-empty, tests pass, `permission_denials` empty), not trust the agent's final message.

3. Is the deterministic (non-AI) backend the right call?

Yes. This is the strongest part of your idea. Choosing which installed agent fills which role is a constraint-satisfaction problem: preferences, availability, capabilities, and cooldown state. It is not a reasoning problem. Anthropic's own docs draw the same line: with subagents and agent teams "Claude is the orchestrator," whereas "a workflow moves the plan into code." claude Deterministic resolution gives you reproducibility, debuggability ("why did it pick Codex for review?") and zero token cost. Keep LLMs inside the stages, not in the scheduler.

Recommended Architecture

Overview

Adapter interface (one per CLI)

Each adapter should implement:

- `detect()`: binary path, version, config directory.
- `authStatus()`: uses the native command where one exists (Claude, Codex, OpenCode) and returns `unknown` otherwise.
- `listModels()`: native command (OpenCode) or a config-declared list.
- `smokeTest(model, timeout)`: a minimal headless prompt with structured output.
- `invoke(task, role, workdir, permissions) → normalized result`.
- `classifyError(stdout, stderr, exitCode, json)`, which returns one of {`not_installed`, `unauthenticated`, `rate_limited`, `quota_exhausted`, `model_unavailable`, `permission_denied`, `timeout`, `ok`}.

Put all of the per-CLI lies and quirks in `classifyError`. For Claude, trust `is_error` and prefer `stream-json` over the exit code. For agy, check `status`. For Gemini, map

exit 41. Ship a fixture test suite of captured real outputs per CLI version. This is the part that rots fastest and the part users will value most.

Also support two generic adapters:

- **ACP:** spawn the agent, call `initialize`, and read `agentCapabilities` and `authMethods`. deepwiki grokipedia This covers capability discovery for the 42 agents in the ACP registry (October 2026 count, per danilchenko.dev) through one code path.
- **Generic command:** user-supplied command template plus a JSON path for the result, as Crewplane does with "any CLI." github

Probing strategy: `--version` is not a health check

Level	What	Cost	Default
L0 Presence	`exec.LookPath`, `--version`, parse semver against an adapter's supported range	~0	Always
L1 Auth	Native status command (Claude `auth status`, `codex login status`, `opencode auth list --format json`); env-var presence for API keys	~0	Always, cached
L2 Functional	One tiny headless turn ("reply with the JSON {"ok":true}") with a hard timeout (e.g. 30–60s)	Tokens and quota	On cache miss, or `--deep`
L3 Model	L2 with each preferred `--model`	Tokens per model	Only for models referenced by roles

Cache results per (adapter, version, auth identity, model) with a TTL: for example 24h for L0, 1h for L1, 15–30 min for L2. Put any target that returns `rate_limited` into a cooldown, as LiteLLM does. litellm Invalidate the cache when the binary version or config file mtime changes. Run probes concurrently with a global timeout so `doctor` finishes in seconds.

Configuration: pick one canonical format

Don't maintain "JSON or YAML" as two first-class formats in your docs. Make YAML canonical and publish a JSON Schema. Because YAML 1.2 is designed as a superset of JSON, a YAML parser will read a `.json` file anyway, so "both" is free at the parser level without doubling your docs. Use the schema for editor autocomplete and for precise validation errors. Avoid adding TOML even though Codex, Agent of Empires and Gemini policies use it; three formats is pure overhead. Example:

```yaml
# agent-doctor.yaml
version: 1
roles:
  primary:   [claude:opus, codex:gpt-5.x, opencode:anthropic/claude-sonnet]
  subagent:  [claude:sonnet, opencode:*]
  auxiliary: [claude:haiku, agy:gemini-flash]
  reviewer:  [codex:*, claude:*]       # prefer a different vendor than prim
agents:
  codex:    { sandbox: workspace-write, profile: ci }
  opencode: { models: [anthropic/claude-sonnet, openai/gpt-5.x-mini] }
policy:
  require_distinct_vendor: [primary, reviewer]
  max_cost_usd_per_run: 5
  probe: { deep: false, ttl_minutes: 30 }
```

All preferences are optional. A missing role falls back to the auto-discovery ranking.

Zero-config auto-discovery

1. Scan PATH for known binaries: `claude`, `codex`, `opencode`, `agy`, `gemini`, `copilot`, `cursor-agent`, `goose`, `aider`, and so on.
2. Check known config dirs (`~/.claude`, `~/.codex`, `~/.config/opencode`, `~/.gemini/antigravity-cli`) and API-key env vars (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY` / `CODEX_API_KEY`, `GEMINI_API_KEY`, `OPENROUTER_API_KEY`).
3. Optionally check the ACP registry for ACP-capable agents.
4. Run L0 and L1 on everything, and L2 only on the top candidate per role.

5. Rank with a static, documented default table (e.g. primary: strongest available frontier agent; auxiliary: cheapest fast model). Print the decision with reasons.

Discovery is cheap. Never silently burn quota: L2 probes in zero-config mode should be opt-in or limited to one per role.

Patterns: express workflows declaratively, and don't invent a new DSL

Model a pattern as a small DAG of stages, each declaring a role (not a vendor), inputs (artifacts from earlier stages), a permission profile, and deterministic gates. Crewplane's "YAML frontmatter for the graph, Markdown for the prompts" is a good format to copy or simply adopt: openorchestrators

```yaml
pattern: plan-implement-review
stages:
  plan:      { role: primary,  permissions: read-only,  output: plan.md }
  implement: { role: subagent, needs: [plan], permissions: workspace-write,
  review:    { role: reviewer, needs: [implement], permissions: read-only,
               gate: { tests: "npm test", diff_nonempty: true, max_loops: 2
```

The resolver binds roles to concrete agent and model pairs at run time from the health manifest, so the same pattern runs on any machine.

Language choice

- **Go (recommended):** a single static binary, excellent subprocess, process-group and timeout handling, trivial cross-compilation, and fast startup for a `doctor` command. Google rewrote Antigravity CLI in Go for similar reasons. agentpedia continuumcode
- **Rust:** equally good, and the choice of Agent of Empires and Vibe Kanban. agent-of-empires virtuslab Pick it if you already know it; there's slower iteration on fiddly output parsers.
- **TypeScript:** best if you want to lean on ACP and the Claude Agent SDK. The cost is that users need Node, and process management is clunkier.
- **Python:** fastest to prototype (Crewplane uses it), but distribution is weaker for a tool that should "just run." pypi

Security

- **Least privilege by default.** Default to read-only and escalate per stage. Codex's headless default is a read-only sandbox, and Gemini's headless mode turns "ask" into "deny." moodybeard dev Don't copy Vibe Kanban's default of permission-skipping flags. elite-ai-assisted-cod…
- **Worktree per writing stage, optional container.** Agent of Empires shows that Docker, Podman or Apple Containers sandboxing with shared auth volumes is practical. hermesatlas agent-of-empires
- **Repo-controlled code runs with your credentials.** Claude Code's `-p` runs repo hooks and `.mcp.json` servers with no trust dialog unless you pass `--bare`. OpenAI warns against job-level `OPENAI_API_KEY` or `CODEX_API_KEY` in workflows that run repo code. dev Pass secrets per invocation through an env allowlist, never log them, and redact probe output (copy Codex doctor's redaction approach). github
- **Verify outcomes deterministically** (tests, diff, `permission_denials`) before marking a stage green.

Recommendations

Verdict: don't build "yet another orchestrator." Build the missing preflight layer, and make it composable.

1. **Phase 1 MVP: `agent-doctor` (about 2–4 weeks part-time).**
   - Adapters for Claude Code, Codex, OpenCode and Antigravity, plus a generic command adapter.
   - L0–L2 probes, error classification, a TTL cache, and cooldowns.
   - `doctor` (human table) and `doctor --json` (capability manifest).
   - YAML config plus JSON Schema, and `resolve` (role → concrete agent/model with an explanation and fallback chain).
   - Zero-config mode.
   - Success criterion: it correctly flags the cases the CLIs themselves misreport (Claude rate-limit exit 0, agy unauthenticated, wrong model name).
2. **Phase 2: integrate rather than compete.** Publish the manifest format. Write a Crewplane provider-selection hook or a Vibe Kanban / Agent of Empires preflight script, or upstream adapters to them. Add an ACP client adapter for broad coverage.

3. **Phase 3 (optional): a thin runner.** Only if the integrations don't fit, add `run <pattern>`: a deterministic DAG executor with worktrees, gates and the role resolver. Keep it under roughly 2k lines and resist adding a UI.
4. **Differentiate on correctness, not breadth.** Version-pinned fixture tests of real CLI outputs, a public compatibility matrix, and fast releases when CLIs change are the actual moat.

Caveats

- **This market moves monthly.**
  - Gemini CLI was replaced by Antigravity CLI for free and individual accounts on June 18, 2026; it remains available through paid API keys and enterprise. aibuilderclub +3
  - OpenCode renamed commands between v1 and v2. computingforgeeks
  - Claude Code ships multiple releases a week. blakecrosley

  Adapter maintenance is the main ongoing cost. Budget for it or don't start.
- **Live probes cost money and quota.** On subscription plans, a probe consumes part of the same rolling 5-hour window you want for real work. fast dev Make deep probes explicit.
- **Some findings come from secondary sources or issue reports, not official docs.** That includes the Codex doctor exit-code behavior, parts of the agy exit-code semantics, and the Claude headless bugs, some of which may already be fixed. Re-verify against the versions you pin.
- **Star counts and adoption figures** (e.g. Ruflo's 710,854 lifetime npm downloads and 45,956 GitHub stars as of May 7, 2026, per Jared Smith's deep dive on sublimecoding.com, and Crewplane's about 41 stars) are self-reported or point-in-time.
- **The strongest argument against building anything:** if you only run Claude Code, its native dynamic workflows plus subagents may be all you need. If you run several CLIs, Crewplane plus a 100-line preflight script may get you 80% of the way. Try both for a week before writing code.